



Pontifícia Universidade Católica de Minas Gerais Instituto de Ciências Exatas e Informática
AEDS III - Trabalho Prático

Sistema de Gerenciamento de Biblioteca

Componentes do Grupo: Caroline Freitas Alvernaz, João de Jesus e Avila, Júlia Rodrigues Melo, Priscila Andrade de Moraes e Sirius Victor Zevallos.

Repositório (GitHub): <https://github.com/prizinh4/biblioteca/tree/main>

Vídeo Explicativo (Youtube): <https://youtu.be/BdkWkdqnVHM?si=IbcUMq1TecsF24wk>

Nota: Alterações para Fase 4

A implementação dos algoritmos de compressão foi feita numa camada específica, alterando minimamente os componentes pré-existentes e sem alterar a lógica de gerenciamento dos dados da biblioteca.

Foi criada uma nova pasta, **compression/**, dentro de **src/**. A mesma contém os arquivos “Huffman.java” e “Lzw.java”, que implementam os algoritmos, e “BackupManager.java” que controla a compressão e descompressão de uma forma centralizada, permitindo selecionar o algoritmo desejado, que cria um arquivo “backup.huff” ou “backup.lzw” no diretório raiz.

Respostas Formulário do Projeto

1 e 2. Qual foi a taxa de compressão obtida com o algoritmo de Huffman e LZW?

a) Tamanho do arquivo original

De acordo com o comando “du -sh” (Disk Usage, summary, human readable) que retorna o tamanho total de um diretório ou arquivo em disco, o conjunto de arquivos armazenados no diretório de dados possuía aproximadamente 76 KB.

```
(base) siriusvlllos@siriusvlllosPC:~/Documentos/PUC/26.1_aedsiii/biblioteca$ du -sh data/  
76K    data/
```

b) Tamanho do arquivo comprimido

De acordo com o mesmo comando, o arquivo de backup gerado pelo algoritmo de Huffman e LZW ambos possuíam aproximadamente 4,0 KB.

```
(base) siriusvllos@siriusvllosPC:~/Documentos/PUC/26.1_aedsiii/biblioteca$ du -sh backup.huff
4,0K    backup.huff
(base) siriusvllos@siriusvllosPC:~/Documentos/PUC/26.1_aedsiii/biblioteca$ du -sh backup.lzw
4,0K    backup.lzw
```

c) Cálculo da Taxa

Taxa de compressão = $((76 - 4) / 76) \times 100$

Taxa de compressão = $(72 / 76) \times 100$

Taxa de compressão = (aprox) 94,74%

d) Interpretação dos resultados

Os algoritmos de Huffman e LZW reduziram o tamanho dos dados de aproximadamente 76 KB para 4 KB, uma redução de aproximadamente de 94,74% do espaço originalmente ocupado.

O resultado indica que os dados armazenados eram redundantes o suficiente para serem representados muito mais eficientemente por meio da codificação baseada na frequência dos caracteres (Huffman) assim como na sequência de caracteres (LZW).

3. Quais dificuldades surgiram ao implementar Huffman e LZW e como você resolveu?

A principal dificuldade foi integrar os algoritmos de compressão à arquitetura, e desenvolver um mecanismo de backup capaz de percorrer o diretório de dados, armazenar o conteúdo dos arquivos em série e reconstruí-los em caso de restauração.

Para mapear os arquivos e conseguir fazer a descompressão, foi necessário armazenar metadados, como um header no começo do arquivo de dados compactados:

```
[quantidade de arquivos (n)]
1: nome, tamanho original
2: nome, tamanho original
. . .
n: nome, tamanho original
[dados compactados]
```

Outro desafio foi adaptar o arquivo BackupManager para suportar múltiplos algoritmos sem ter duplicação de código, e a solução adotada foi centralizar a lógica de leitura e escrita dos arquivos em um único componente, permitindo que os métodos de Huffman ou LZW fossem selecionados na etapa de compressão/descompressão.

Documentação do Projeto

1. Descrição do Problema

O sistema visa gerenciar as operações de uma biblioteca, permitindo o controle através do CRUD de Livros, Leitores, Exemplares e Reservas. Os dados são armazenados em arquivos binários no HD, sem SGBD, com cabeçalho de controle e lápide para exclusão lógica.

2. Objetivos do Trabalho

- Desenvolver um sistema que permita o CRUD de Livros, Leitores, Autores e Empréstimos.
- Garantir persistência direta em arquivos binários no HD com controle de exclusão lógica (lápide).
- Implementar relacionamentos 1:N e N:N e utilizar diversos tipos de dados (Data, Real, String e String Multivalorada).

3. Requisitos Funcionais

- RF01: Cadastrar, consultar, atualizar e excluir (CRUD) Livros, considerando associação entre Livros e Exemplares.
- RF02: Cadastrar, consultar, atualizar e excluir (CRUD) Leitores.
- RF03: Gerenciar informações de autores e categorias integradas ao cadastro de livros.
- RF04: Registrar e gerenciar Empréstimos de exemplares de livros para os leitores.
- RF05: Listar registros ativos de qualquer entidade.
- RF06: Cadastrar, consultar, atualizar e excluir (CRUD) de Exemplares de Livros, onde cada exemplar representa uma instância física de um determinado livro.
- RF07: Consulta de Exemplares por Livros, permitindo listagem de todos os exemplares associados a um determinado livro.
- RF08: Criar, cancelar e listar Reservas (N:N entre Leitor e Livro)
- RF10: Listar reservas de um leitor
- RF11: Listar reservas de um livro

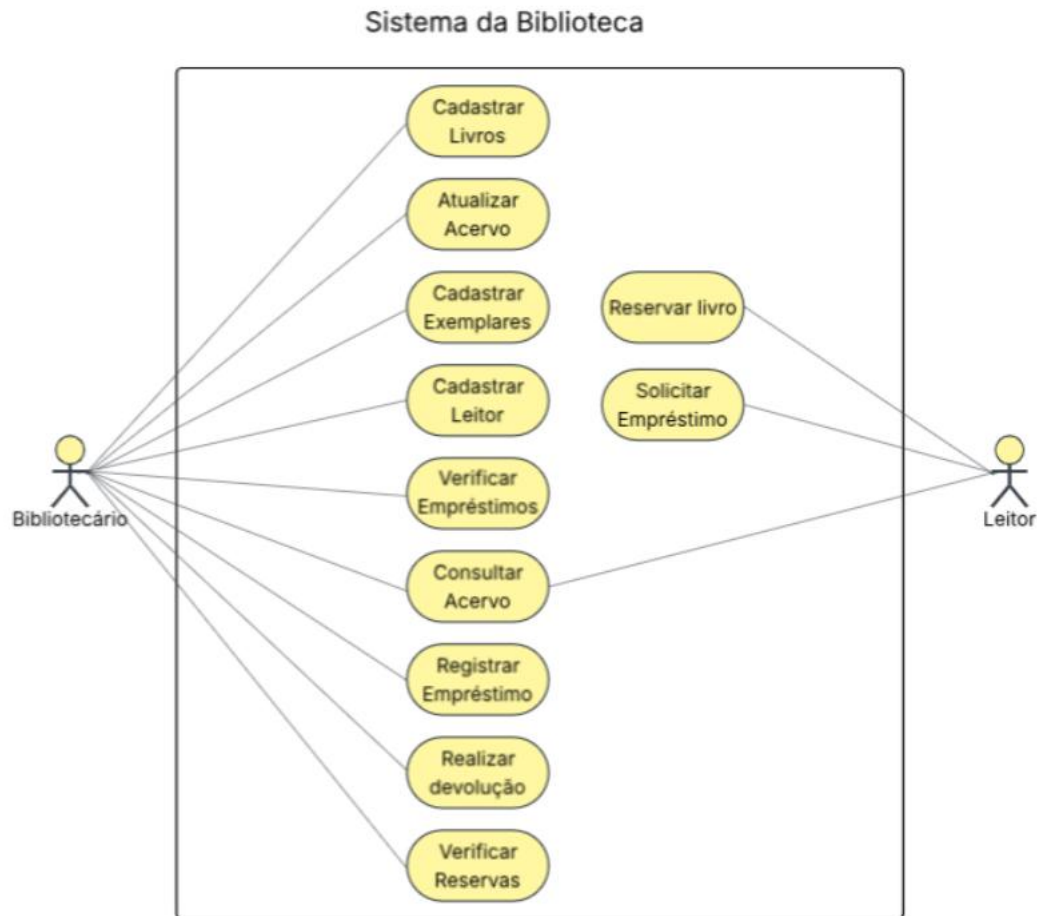
4. Requisitos Não Funcionais

- RNF01: O backend deve ser implementado persistindo dados em arquivos binários.
- RNF02: A interface final (após a Fase 1) não poderá utilizar console, devendo possuir interface mínima em HTML/CSS (Front-end web ou interface gráfica).
- RNF03: O sistema deve seguir o padrão de arquitetura MVC + DAO.
- RNF04: O sistema deve garantir acesso eficiente aos dados por meio de estruturas de indexação (índices diretos, hash extensível e árvore B+).
- RNF05: O sistema deve garantir validação da FK Livro na inserção de Exemplares, exclusão lógica em cascata e consistência em atualizações do relacionamento.

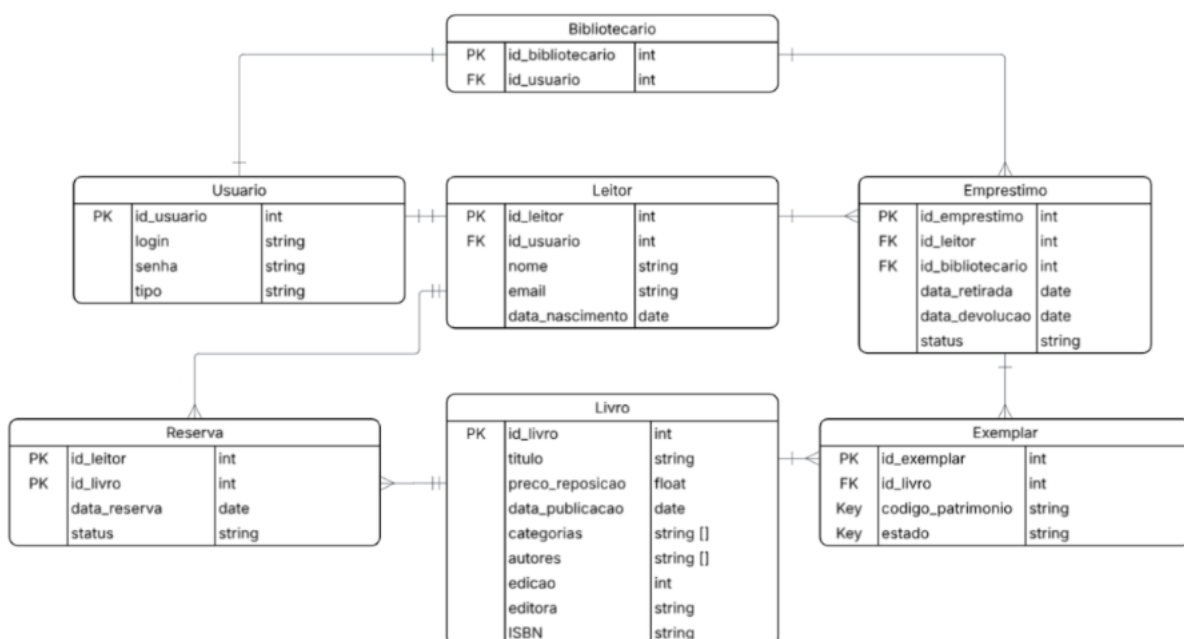
5. Atores

- Bibliotecário (Administrador): Realiza a gestão do acervo (Livros e Autores) e gerencia os cadastros de Leitores.
- Leitor: Usuário final que realiza o empréstimo e reservas de livros.

6. Diagrama de Caso de Uso



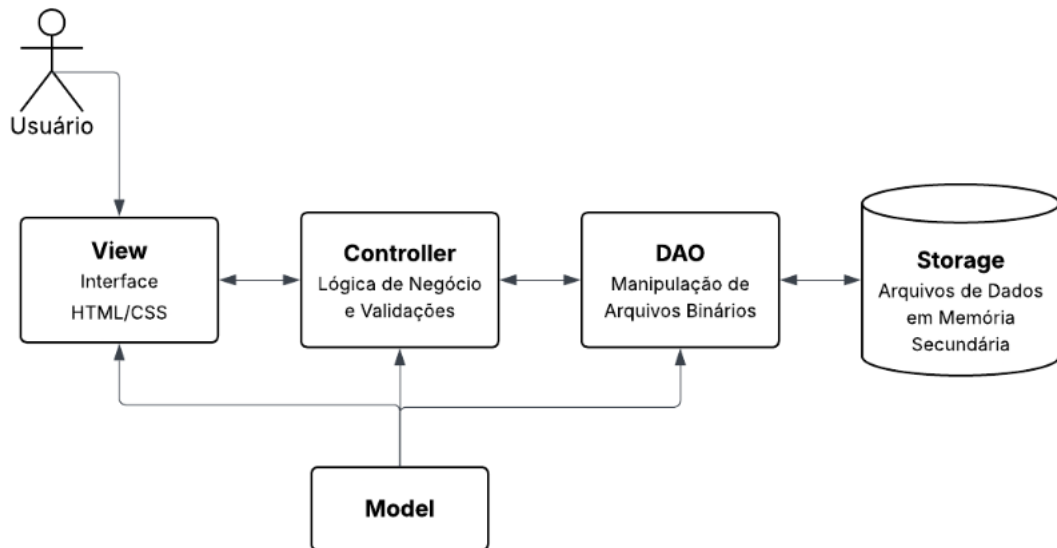
7. Diagrama Entidade-Relacionamento



8. Arquitetura Proposta

O sistema seguirá o padrão MVC + DAO, onde:

- **Model:** Classes de domínio do sistema (Livro, Leitor, Autor, Empréstimo).
- **DAO:** Classes responsáveis por ler e escrever diretamente nos arquivos binários (.db), manipulando o cabeçalho e a exclusão lógica (lápide).
- **Controller:** Classes que gerenciam as regras de negócio (ex: não permitir empréstimo se o leitor tiver multas).



View: Interface gráfica ou web (HTML/CSS) para interação com o usuário.

O sistema utiliza uma arquitetura em camadas para garantir a separação de responsabilidades e facilitar a manutenção:

- **View:** Camada de interação com o usuário. Inicialmente implementada via Interface Web (HTML/CSS), captura as entradas do usuário e exibe os resultados.
- **Controller:** Atua como intermediária. Ela recebe as solicitações da View, aplica as regras de negócio (como verificar se um livro está disponível antes de realizar o empréstimo) e coordena as chamadas para a camada de persistência.
- **DAO (Data Access Object):** Camada responsável exclusivamente pelo acesso aos dados. É nela que será implementada a lógica de RandomAccessFile para abrir os arquivos .db, gerenciar o ponteiro do arquivo (seek), ler/escrever o cabeçalho e realizar a exclusão lógica via lápide.
- **Model:** Contém as classes de domínio (Livro, Leitor, Exemplar, Empréstimo, Usuário). Essas classes representam a estrutura dos dados que são transferidos entre as camadas e espelham os registros armazenados nos arquivos binários.
- **Persistência:** Os dados são armazenados diretamente no sistema de arquivos do computador em formato binário, garantindo a permanência das informações sem o uso de bancos de dados externos.